

Chapitre : Génération de code Objet

La Génération de code produite par les phases précédentes a pour but d'engendrer le texte objet. Elle a pour donnée un programme compilé et produit comme résultat un programme cible équivalent.

La qualité du code produit est déterminée par sa vitesse d'exécution et sa taille. Une machine cible possédant un jeu d'instructions riche peut fournir plusieurs façons d'implémenter une opération donnée.

Une bonne connaissance de la machine cible et de son jeu d'instructions est un préalable à la conception d'un bon générateur de code.

Un générateur de code doit satisfaire les points suivants :

- Doit être correct.
- De bonne qualité.
- Il doit s'exécuter le plus rapidement possible.

Alors que certains détails d'implémentation dépendent de la machine cible et du système d'exploitation, des problèmes tels que la gestion de la mémoire, la sélection des instructions, l'allocation de registres et l'ordre d'évaluation sont inhérents à tous les générateurs de code.

Plusieurs choix de forme intermédiaire sont utilisés, forme postfixée, quadruplets, ...
. Dans notre cours, nous utilisons les quadruplets.

La machine cible utilisée est à 8 registres. Les opérations arithmétiques et logiques se font dans l'accumulateur.

Instructions de base

a. Chargement

LOAD M	Mettre le contenu de M dans Acc
LREG i, M	(M) dans reg(i)
LACCR i	(i) dans ACC
LR ACC i	(ACC) dans reg i

b. Rangement

STORE M	(ACC) dans M
SREG i, M	(i) dans M

c. Pour les opérations

ADD (ADDF) M	$(ACC) + (M) \rightarrow (ACC)$
SUB (SUB F)	$(ACC) - (M) \rightarrow (ACC)$
MULT (MULT F)	$(ACC) * (M) \rightarrow (ACC)$
DIV (DIV F)	$(ACC) / (M) \rightarrow (ACC)$
ABS	Prendre la valeur absolue du (ACC)
CHS	Changer le signe
B M	Faire un saut à l'adresse M

Génération de code à partir de quadruplets

- . La gestion des temporaires vise à minimiser le nombre de temporaires utilisés par le programme.
- . La gestion des registres vise à assurer la pleine utilisation des registres et minimiser le nombre de changement et de rangement de ces registres.
- . Avant de ranger dans l'accumulateur, prendre le soin de ne pas écraser son contenu.
- . Initialement, le contenu de l'accumulateur est vide.
- . Pour générer le chargement du 1er opérande dans l'accumulateur, la procédure GetinACC (X, Y) est utilisée.
- . On définit une variable acc qui contrôle l'état de l'accumulateur à l'exécution. Cette variable est définie à la compilation.

```
Procédure GetinACC (X, Y)    /* met X ou Y dans ACC */
begin
  if acc = ' '               /* initialement ACC est vide */
  then begin  Gen ('LOAD ', X)
              acc := X
              end;

  if acc = 'Y'               /* l'accumulateur contient le 2ième opérande */
  then begin  T := X
              X := Y          /* permutation dans le quadruplet */
              Y := T
  else if acc = 'X' then begin Gen ('STORE ', Z )
                              Gen ('LOAD ', X);  acc := X
                              end;
```

Génération de code pour '+'

```
GetinACC (QD (i,2), QD(i, 3))
Gen ('ADD', QD(i, 3));
acc := QD(i,4)
```

L'appel de Getinacc avec les paramètres QD(i,2) et QD(i,3) va réaliser :

- if acc = ' ' then gen ('LOAD', QD(i,2));
- if acc = QD(i,3) then 'permutation , + commutatif';
- if (acc QD(i,3) and (acc QD(i, 2) then Gen ('STORE' , Z)
Gen ('LOAD', QD(i, 2))
acc := QD (i, 2)

Génération pour '*'

Getinacc (QD(i,2), QD(i,3))
Gen ('MULT', QD(i,3));
acc := QD(i,4)

Génération pour '-'

Getinacc (QD(i,2), ' ')
Gen ('SUB', QD (I, 3));
acc := QD(i,4)

Remarque : Le '-' binaire n'étant pas commutatif, le chargement du 1er opérande doit se faire, si acc est différent du 1er opérande (pas permutation).

Génération pour le '-' unaire

Getinacc (QD(i,2), ' ')
Gen ('CHS', ' ');
acc := QD(i,4)

Exemple : Soit l'expression $a * ((a * b * c) - (c * (-b)))$

Getinacc	acc	code
	vide	
Getinacc ('a','b')	a	Load a
	T1	Mult b
Getinacc ('T1', c)	T2	Mult c
Getinacc ('b', ' ')		Store T2
	b	Load b
	T3	CHS
Getinacc ('c', T3) permutation des opérandes dans le quadruplets		
(* , T3, c,T4)	T4	
Getinacc ('T2', ' ')		Store T4
		Load T2
	T5	Sub T4
Getinacc ('a', T5) permutation des opérandes dans le quadruplet		
	T6	Mult a

